CS 6515: Graduate Algorithms

Lecture 1: Course Overview, Asymptotic Notation, and Merge Sort

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. PhD students on waitlist should email Sahil (ssingla@gatech.edu) their gtid, email, and PhD program name.
2. HW1 will be released by tomorrow. It will be due Sep 3.

2 What is this course about?

An *algorithm* is a sequence of instructions that accomplishes a task. We usually want more than correctness: we also want to minimize the resources used, such as time, space, or communication.

1. There is no single idea that solves every algorithmic problem. Instead, we will develop general approaches that apply to broad classes of problems;
2. learn useful abstractions and mathematical/proof techniques;
3. develop a theoretical way of thinking (a.k.a. theoretical lens) about new questions: useful even beyond computer science in fields, e.g., in fields such as economics, mathematics, and physics.

During the course, you are encouraged to use AI to learn and self-evaluate. The goal is to be curious, always learning, and develop skills to learn new things quickly.

Course roadmap

Part I Divide and conquer; dynamic programming; graph algorithms.

Part II Linear programming; convex optimization and gradient descent; approximation algorithms.

Part III Randomized algorithms; complexity theory.

Most communication, homework, and class notes will be posted on Piazza. Please consult the current syllabus on Canvas for grading policies, office hours, and other administrative details.

3 Asymptotic notation

For an input of size n, let $T(n)$ (or more generally $f(n)$) denote the amount of a resource used by an algorithm. We typically count *elementary operations* (like addition, subtraction, multiplication, and bit operations) and focus on how this quantity grows as the integer n becomes large.

3.1 Big-$\mathcal{O}$ notation: asymptotic upper bounds

Definition (Big-$\mathcal{O}$). Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We write

$$f(n) = \mathcal{O}(g(n))$$

if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq c\, g(n) \qquad \text{for every } n \geq n_0.$$

Thus, g is an eventual upper bound on f, up to a constant factor. When the limit exists, a useful test is

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty \quad \Longrightarrow \quad f(n) = \mathcal{O}(g(n)).$$

Example. We have $n^2 + 2n = \mathcal{O}(n^2)$: for $n \geq 1$, $n^2 + 2n \leq 3n^2$. We also have $n^2 + 2n = \mathcal{O}(n^3)$. Big-$\mathcal{O}$ is an upper bound, not necessarily the tightest upper bound.

3.2 Lower bounds, strict bounds, and tight bounds

Definition (Big-Ω). We write $f(n) = \Omega(g(n))$ if $g(n) = \mathcal{O}(f(n))$. Equivalently, there are constants $c > 0$ and n_0 such that $f(n) \geq c\, g(n)$ for every $n \geq n_0$.

Definition (Little-o and Little-ω). We write $f(n) = o(g(n))$ if, for every constant $c > 0$, there is an n_0 such that $f(n) < c\, g(n)$ for every $n \geq n_0$. For positive g, this is equivalent to

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$

Intuitively, f grows strictly more slowly than g.

Similarly, $f(n) = \omega(g(n))$ means $g(n) = o(f(n))$.

Definition (Big-Θ). We write $f(n) = \Theta(g(n))$ if both $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$.

Some useful comparisons are

$$1 = o(\log\log n) = o(\log n) = o(n^{0.1}) = o(n) = o(2^n) = o(2^{n^2}).$$

Also, $n! = o(n^n)$, since

$$\frac{n^n}{n!} = \prod_{i=1}^{n} \frac{n}{i} \longrightarrow \infty.$$

Exercise. Prove that

$$\log(n!) = \Theta(n \log n).$$

One route is to compare $n!$ with n^n for the upper bound and to use only the largest $n/2$ factors in the product for the lower bound.

4 Divide and Conquer

The divide-and-conquer paradigm has three steps:

1. **Divide:** break the problem into smaller subproblems.
2. **Conquer:** solve each subproblem recursively.
3. **Combine:** assemble the subproblem solutions into a solution to the original problem.

5 Case study: merge sort

Input. An array $A[0], A[1], \ldots, A[n-1]$ of n numbers. (Here we are ignoring bit complexity; so imagine they are all 64-bit numbers and can be compared/rearranged in $\mathcal{O}(1)$ time.)

Goal. Rearrange the numbers in nondecreasing order.

For simplicity, assume that n is a power of two. This assumption is not essential: one may split using floors and ceilings, or conceptually pad the array.

5.1 Algorithm

1. If $n \leq 1$, return A.
2. Divide A into its left and right halves.
3. Recursively sort each half.
4. Merge the two sorted halves.

The merge step repeatedly compares the first unmerged element of each half, moves the smaller element to the output, and finally appends the unused suffix of the nonempty half.

Exercise. Show that two sorted arrays, each of length n, can be merged into one sorted array of length $2n$ in $\mathcal{O}(n)$ time.

Correctness. Correctness follows by induction on n. We will discuss induction in detail in next lecture, but briefly: The base case $n \leq 1$ is immediate. For the inductive step, the recursive calls correctly sort the two halves by the inductive hypothesis. The merge procedure outputs their elements in nondecreasing order and outputs every input element exactly once. Therefore the result is a sorted permutation of the original array.

5.2 Running time

Let $T(n)$ denote the worst-case running time on an input of length n. The two recursive calls operate on inputs of size $n/2$, and the merge step takes cn time for some constant $c > 0$. Thus

$$T(n) = 2T(n/2) + cn, \qquad T(1) = \Theta(1).$$

We can solve this recurrence using a recursion tree. At level i, there are 2^i subproblems, each of size $n/2^i$:

Level	Number of subproblems	Merge work per subproblem	Total merge work
0	1	cn	cn
1	2	$cn/2$	cn
2	4	$cn/4$	cn
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$cn/2^i$	cn

The subproblem size reaches 1 after $\log_2 n$ levels. Every non-leaf level does cn work, and the n leaves contribute $\Theta(n)$ total work. Hence

$$T(n) = cn \log_2 n + \Theta(n) = \Theta(n \log n).$$

Takeaway. Divide and conquer can turn a problem into many simpler copies of itself. Its efficiency depends on both the size and number of the subproblems and on the cost of combining their solutions.

CS 6515: Graduate Algorithms

Lecture 2: Induction and Divide-and-Conquer

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW 1 available on Piazza; due next Thursday on Gradescope.
2. Lecture notes uploaded to Piazza.
3. Office hours starting Monday.

2 Recap and plan

Last time we introduced asymptotic notation and merge sort. Its running time satisfies

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + cn, \qquad T(1) = 1,$$

often written more simply as $T(n) = 2T(n/2) + cn$. A recursion tree suggests $T(n) = \Theta(n \log n)$.

Today we will:

- verify recurrence bounds using induction;
- record a useful form of the Master Theorem;
- design a linear-time algorithm for the maximum-difference problem; and
- find the median in deterministic linear time.

3 Proving a recurrence bound by induction

The general recipe is simple: use a recursion tree, experimentation, or another heuristic to *guess* a bound, and then prove that bound by induction.

We use *strong induction* on the integer input size: when proving a bound for $T(n)$, the induction hypothesis is available for every smaller integer. This removes the need to restrict n to powers of two and handles unequal recursive subproblem sizes naturally.

Theorem (merge-sort recurrence). Suppose $c > 0$, $T(1) > 0$ is a constant, and for every integer $n \geq 2$,

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + cn.$$

Then $T(n) = \mathcal{O}(n \log n)$.

Proof. All logarithms in this proof are base two. We will use strongly induction.

Induction hypothesis: $T(m) \le Km \log_2 m$ for every integer $m \ge 2$.

Base case: Choose K sufficiently large so that the claim holds for $m = 2, 3$ and $K \log_2(3/2) \ge c$.

Induction step: For $n \ge 4$, set $a = \lfloor n/2 \rfloor$ and $b = \lceil n/2 \rceil$. Then $a + b = n$, both a and b are smaller than n, and

$$a, b \le \frac{2n}{3}.$$

The strong induction hypothesis therefore gives

$$\begin{aligned} T(n) &\le K(a \log_2 a + b \log_2 b) + cn \\ &\le K(a+b) \log_2(2n/3) + cn \\ &= Kn \log_2 n + Kn \log_2(2/3) + cn \\ &\le Kn \log_2 n. \end{aligned}$$

Thus $T(n) = \mathcal{O}(n \log n)$. □

Exercise. Prove the matching lower bound $T(n) = \Omega(n \log n)$ by strong induction. One useful observation is that $\lfloor n/2 \rfloor, \lceil n/2 \rceil \ge n/3$ for $n \ge 2$. Together with the theorem, this gives $T(n) = \Theta(n \log n)$.

The next example previews the recurrence arising from deterministic median finding. Unlike merge sort, its two recursive calls have different sizes, so no convenient powers-of-two assumption would help.

Theorem (selection-type recurrence). Suppose $c > 0$, T is bounded on constant-size inputs, and for all sufficiently large integers n,

$$T(n) \le T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn.$$

Then $T(n) = \mathcal{O}(n)$.

Proof. We use strong induction.

Induction hypothesis: $T(m) \le Am$ for every integer $m \le n$.

Base case: Choose $A \ge 20c$ and enlarge it, if needed, so that the claim holds for the finitely many small base cases.

Induction step: For n sufficiently large, both recursive arguments are smaller than n, and

$$\begin{aligned} T(n) &\le A\lceil n/5\rceil + A\big(\lceil 7n/10\rceil\big) + cn \\ &\le \frac{9}{10}An + 2A + cn \\ &\le \frac{19}{20}An + 2A \\ &\le An. \end{aligned}$$

The last inequality holds for large n; all smaller values were included among the base cases. □

3.1 A useful form of the Master Theorem

Consider constants $a \ge 1$, $b > 1$, and $d \ge 0$, and the recurrence

$$T(n) = aT(n/b) + \Theta(n^d), \qquad T(1) = \Theta(1).$$

Suppressing floors, ceilings, and constant-size rounding effects, the solution is

$$T(n) = \begin{cases} \Theta(n^d), & d > \log_b a, \\ \Theta(n^d \log n), & d = \log_b a, \\ \Theta\big(n^{\log_b a}\big), & d < \log_b a. \end{cases}$$

This compares the work n^d performed at the root with the total growth $n^{\log_b a}$ arising from the recursive leaves.

4 Maximum difference

Input. Numbers $A[0], A[1], \ldots, A[n-1]$. (Again assume that these numbers are represented using constant number of bits.)

Goal. Find indices $i \le j$ that maximize

$$A[i] - A[j].$$

In other words, find the largest drop from an earlier element to a later one.

Example. For

$$A = [2, 5, 1, 7, 3, 2, 4, 6],$$

an optimal value is $7 - 2 = 5$, attained at $i = 3$ and $j = 5$.

4.1 Approach 1: return only the best difference

Split the array into left and right halves and recursively compute the best difference within each half. An optimal pair is one of three types:

1. both indices lie in the left half;
2. both indices lie in the right half; or
3. i lies in the left half and j lies in the right half.

If a recursive call returns only the best difference, computing the crossing solution requires scanning the two halves to find

$$\max(A_{\text{left}}) - \min(A_{\text{right}}).$$

Thus the recurrence is

$$T(n) = 2T(n/2) + \mathcal{O}(n) = \Theta(n \log n).$$

4.2 Approach 2: return a richer summary

Each recursive call returns three quantities:

$$D\text{: maximum difference,} \qquad M\text{: maximum element,} \qquad m\text{: minimum element.}$$

For a single element x, return $(0, x, x)$. Given summaries (D_L, M_L, m_L) and (D_R, M_R, m_R) for the two halves, combine them using

$$\begin{aligned} D &= \max\{D_L, D_R, M_L - m_R\}, \\ M &= \max\{M_L, M_R\}, \\ m &= \min\{m_L, m_R\}. \end{aligned}$$

All three quantities take constant time to combine. Therefore

$$T(n) = 2T(n/2) + \mathcal{O}(1) = \Theta(n).$$

Why it works. Every feasible pair lies wholly in the left half, wholly in the right half, or crosses from the left half to the right half. The three candidates in the formula for D are precisely the best solutions of these three types.

Exercise. Prove $T(n) = \Theta(n)$ from the recurrence, both by induction and with a recursion tree. At level r, how much total combine work is performed?

5 Selection and median finding

Input. Distinct numbers $A[0], \ldots, A[n-1]$ and a rank $k \in \{1, 2, \ldots, n\}$.

Goal. Find the k-th smallest element. It has exactly $k-1$ smaller elements and $n-k$ larger elements.

Sorting first takes $\Theta(n \log n)$ time. The selection algorithm of Blum, Floyd, Pratt, Rivest, and Tarjan achieves deterministic linear time by finding a provably good pivot.

Theorem (deterministic selection). The k-th order statistic of n numbers can be found in $\mathcal{O}(n)$ time.

5.1 Proof plan: what makes a pivot good?

Suppose we choose a pivot p and partition the input into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Once we know $|L|$, we can determine whether the desired element is p, lies in L, or lies in G. We then recurse on only the relevant side.

An arbitrary pivot need not help: if it is the smallest or largest element, the recursive problem can still have size $n-1$. Repeated linear-time partitioning would then lead to the recurrence

$$T(n) \leq T(n-1) + cn,$$

which permits quadratic running time. We therefore need a pivot that is bounded away from both extremes; it need not be the exact median.

Definition (good pivot). We call p a *good pivot* if both L and G have size at most

$$\frac{7n}{10} + \mathcal{O}(1).$$

Equivalently, up to constant rounding terms, at least $3n/10$ elements lie on each side of p and can be discarded when we recurse on the other side.

Our proof will follow three steps:

1. In $\mathcal{O}(n)$ nonrecursive work, form about $n/5$ representative elements, then recursively select their median. This costs $T(\lceil n/5 \rceil) + \mathcal{O}(n)$ and produces the pivot p.
2. Prove that this particular p is a good pivot.
3. Partition around p in $\mathcal{O}(n)$ time and recurse on at most $7n/10 + \mathcal{O}(1)$ elements.

Consequently, the running time will satisfy the recurrence previewed in Section 2:

$$T(n) \leq T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn = \mathcal{O}(n).$$

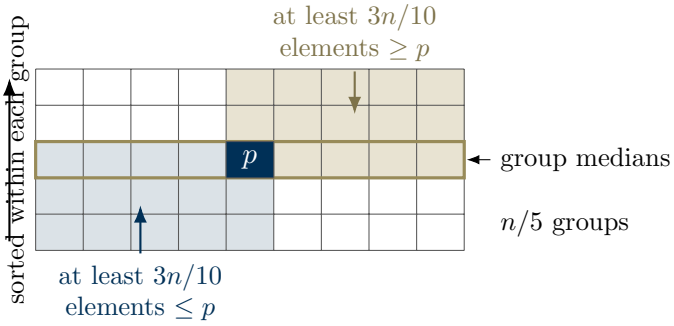
The construction below is designed precisely to establish the missing good-pivot guarantee.

5.2 Finding a good pivot

For clarity, first suppose n is divisible by 5 and ignore harmless rounding terms.

1. Partition the input into $n/5$ groups of five.

2. Sort each group and record its median. Because every group has constant size, this takes $\mathcal{O}(n)$ total time.
3. Recursively find the median p of the $n/5$ group medians.



Columns are shown ordered by their medians only to visualize the counting argument.

Claim. The median-of-medians pivot p has at least $3n/10$ elements no smaller than it and at least $3n/10$ elements no larger than it, up to constant rounding terms.

Proof of the claim. At least half of the group medians are at least p. In each corresponding group, the median and the two larger elements are all at least p. Hence at least

$$3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$$

elements are at least p. The symmetric argument gives at least $3n/10$ elements at most p. □

5.3 Discarding one side

Partition the input in $\mathcal{O}(n)$ time into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Then:

- if $k \leq |L|$, recurse on L with rank k;
- if $k = |L| + 1$, return p;
- if $k > |L| + 1$, recurse on G with rank $k - |L| - 1$.

Because p is a good pivot, the recursive side contains at most $7n/10 + \mathcal{O}(1)$ elements.

5.4 Running time

The recursive call that finds the pivot uses an input of size about $n/5$, and the recursive selection call uses an input of size at most $7n/10 + \mathcal{O}(1)$. Grouping, sorting the five-element groups, and partitioning cost $\mathcal{O}(n)$. Thus, ignoring rounding terms,

$$T(n) \leq T(n/5) + T(7n/10) + cn.$$

Exercise. Include the rounding terms and match this recurrence to the selection-type theorem from Section 2. Conclude that deterministic selection runs in $\mathcal{O}(n)$ time.

Takeaway. Divide-and-conquer algorithms become faster when recursive calls return exactly the extra information needed for a cheap combine step, or when a good pivot lets us discard a constant fraction of the input at every step.

CS 6515: Graduate Algorithms

Lecture 3: From Divide-and-Conquer to Dynamic Programming

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW 1 is due this Thursday (Sep 3).
2. Office Hours schedule available on Piazza.
3. Plan for today: Finish median finding in linear time and discussion on Divide-and-conquer. Start Dynamic Programming and discuss Largest Contiguous Sum and Longest Common Subsequence problems.

2 Linear Time Median finding

Input. Distinct numbers $A[0], \ldots, A[n-1]$ and a rank $k \in \{1, 2, \ldots, n\}$.

Goal. Find the k-th smallest element. It has exactly $k-1$ smaller elements and $n-k$ larger elements.

Sorting first takes $\Theta(n \log n)$ time. The selection algorithm of Blum, Floyd, Pratt, Rivest, and Tarjan achieves deterministic linear time by finding a provably good pivot.

Theorem (deterministic selection). The k-th order statistic of n numbers can be found in $\mathcal{O}(n)$ time.

2.1 Proof plan: what makes a pivot good?

Suppose we choose a pivot p and partition the input into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Once we know $|L|$, we can determine whether the desired element is p, lies in L, or lies in G. We then recurse on only the relevant side.

An arbitrary pivot need not help: if it is the smallest or largest element, the recursive problem can still have size $n-1$. Repeated linear-time partitioning would then lead to the recurrence

$$T(n) \leq T(n-1) + cn,$$

which permits quadratic running time. We therefore need a pivot that is bounded away from both extremes; it need not be the exact median.

Definition (good pivot). We call p a *good pivot* if both L and G have size at most

$$\frac{7n}{10} + \mathcal{O}(1).$$

Equivalently, up to constant rounding terms, at least $3n/10$ elements lie on each side of p and can be discarded when we recurse on the other side.

Our proof will follow three steps:

1. In $\mathcal{O}(n)$ nonrecursive work, form about $n/5$ representative elements, then recursively select their median. This costs $T(\lceil n/5 \rceil) + \mathcal{O}(n)$ and produces the pivot p.
2. Prove that this particular p is a good pivot.
3. Partition around p in $\mathcal{O}(n)$ time and recurse on at most $7n/10 + \mathcal{O}(1)$ elements.

Consequently, the running time will satisfy the recurrence previewed in Section 2:

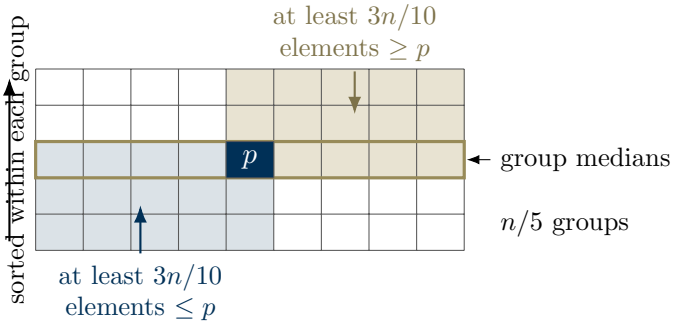
$$T(n) \leq T(\lceil n/5 \rceil) + T(\lceil 7n/10 \rceil) + cn = \mathcal{O}(n).$$

The construction below is designed precisely to establish the missing good-pivot guarantee.

2.2 Finding a good pivot

For clarity, first suppose n is divisible by 5 and ignore harmless rounding terms.

1. Partition the input into $n/5$ groups of five.
2. Sort each group and record its median. Because every group has constant size, this takes $\mathcal{O}(n)$ total time.
3. Recursively find the median p of the $n/5$ group medians.



Columns are shown ordered by their medians only to visualize the counting argument.

Claim. The median-of-medians pivot p has at least $3n/10$ elements no smaller than it and at least $3n/10$ elements no larger than it, up to constant rounding terms.

Proof of the claim. At least half of the group medians are at least p. In each corresponding group, the median and the two larger elements are all at least p. Hence at least

$$3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$$

elements are at least p. The symmetric argument gives at least $3n/10$ elements at most p. □

2.3 Discarding one side

Partition the input in $\mathcal{O}(n)$ time into

$$L = \{x : x < p\}, \qquad \{p\}, \qquad G = \{x : x > p\}.$$

Then:

- if $k \le |L|$, recurse on L with rank k;
- if $k = |L| + 1$, return p;
- if $k > |L| + 1$, recurse on G with rank $k - |L| - 1$.

Because p is a good pivot, the recursive side contains at most $7n/10 + \mathcal{O}(1)$ elements.

2.4 Running time

The recursive call that finds the pivot uses an input of size about $n/5$, and the recursive selection call uses an input of size at most $7n/10 + \mathcal{O}(1)$. Grouping, sorting the five-element groups, and partitioning cost $\mathcal{O}(n)$. Thus, ignoring rounding terms,

$$T(n) \le T(n/5) + T(7n/10) + cn.$$

Exercise. Include the rounding terms and match this recurrence to the selection-type theorem from Section 2. Conclude that deterministic selection runs in $\mathcal{O}(n)$ time.

Divide-and-conquer takeaway. We divide an instance into smaller, largely separate subproblems, solve them recursively, and combine their answers. The quality of the split and the cost of the combine step determine the recurrence. Merge sort, maximum difference, and deterministic selection illustrate three different recurrence shapes.

3 Dynamic programming

Dynamic programming (DP) is useful when a problem has many smaller *overlapping* subproblems. Instead of resolving the same subproblem many times, we store its answer and reuse it.

Most DP algorithms follow four design steps:

1. define a collection of subproblems, often index by non-negative integers $i, j, \ldots$;
2. give a recurrence relating each subproblem to smaller ones, i.e., if we have solutions to the subproblems (like stored in an array $M[i, j, \ldots]$) then we can solve the original problem;
3. identify base cases and an evaluation order;
4. recover the desired solution, not merely its value, if needed.

4 Largest contiguous sum

Input. An array $A[0], A[1], \ldots, A[n-1]$ of numbers (possibly negative). Again assume that each number uses $\mathcal{O}(1)$ bits.

Goal. Find an interval $[x, y]$ maximizing

$$\sum_{i=x}^{y} A[i].$$

We permit the empty interval, whose value is 0.

Example. For

$$A = [5, 15, -30, 10, -5, 40, 10],$$

the interval $[10, -5, 40, 10]$ has value 55 and is optimal.

The key is to define a subproblem that remembers enough information to extend a solution by one position.

Definition (subproblem). For $0 \le i < n$, let $M[i]$ be the largest sum of a contiguous subarray that ends exactly at position i.

Any nonempty interval ending at i either begins at i, or extends the best interval ending at $i-1$. Therefore

$$M[0] = A[0], \qquad M[i] = \max\{A[i],\ A[i] + M[i-1]\} \quad (i \ge 1).$$

The desired value is

$$\max\left\{0, \max_{0 \le i < n} M[i]\right\}.$$

There are n states and each takes constant time, so the algorithm runs in $\mathcal{O}(n)$ time. Only the preceding value is needed if we want the optimum value alone, giving $\mathcal{O}(1)$ auxiliary space.

Exercise. Fill in the values $M[i]$ for the example. Then augment the algorithm so that it returns the two endpoints of an optimal interval.

Correctness invariant. After computing $M[i]$, it equals the best sum among all intervals ending at i. This follows by induction: removing $A[i]$ from an interval that starts before i leaves an interval ending at $i-1$, and the recurrence chooses the better of extending that interval and starting anew.

5 Longest common subsequence

Input. Sequences

$$A = a_1 a_2 \cdots a_n, \qquad B = b_1 b_2 \cdots b_m.$$

A *subsequence* is obtained by deleting zero or more elements without changing the order of those retained. Unlike a substring, its elements need not occupy consecutive positions.

Goal. Find the maximum length of a sequence that is a subsequence of both A and B.

Example. If $A = \texttt{ABCDEFG}$ and $B = \texttt{BCDGKAB}$, then `BCDG` is a common subsequence of length 4.

Definition (prefix subproblem). For $0 \le i \le n$ and $0 \le j \le m$, let $D[i, j]$ be the length of a longest common subsequence of $a_1 \cdots a_i$ and $b_1 \cdots b_j$.

The empty prefix gives the base cases

$$D[0, j] = D[i, 0] = 0.$$

For $i, j \ge 1$,

$$D[i, j] = \begin{cases} D[i-1, j-1] + 1, & a_i = b_j, \\ \max\{D[i-1, j], D[i, j-1]\}, & a_i \neq b_j. \end{cases}$$

Why the recurrence works. If $a_i = b_j$, some longest common subsequence can end with this common symbol; what precedes it is an LCS of the two shorter prefixes. If $a_i \neq b_j$, a common subsequence cannot use both final symbols, so at least one must be discarded.

The table has $(n + 1)(m + 1)$ entries, each filled in constant time. Thus the running time is $\mathcal{O}(nm)$ and the straightforward space usage is $\mathcal{O}(nm)$.

5.1 Recovering an actual LCS

Start at (n, m) and trace backward:

- if $a_i = b_j$, output that symbol and move to $(i - 1, j - 1)$;
- otherwise move to a neighboring state whose value equals $D[i, j]$.

The symbols are discovered in reverse order. The traceback takes $\mathcal{O}(n + m)$ time once the table is available.

Exercise. Draw the DP table for the two strings in the example and recover one LCS. Where can ties occur, and what do they mean?

DP takeaway. The hardest part is usually not filling the table; it is choosing a state that contains exactly the information needed by the recurrence. Next time we will practice this idea on trees and on the knapsack problem.

CS 6515: Graduate Algorithms

Lecture 4: Tree Dynamic Programming, Knapsack, and an FPTAS

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW 1 due today.
2. HW 2 out in a couple of days (due on Thurs Sep 17).

2 Recap and plan

Last time we used dynamic programming for the largest contiguous sum. The central design question was:

What information must a subproblem remember so that it can be extended?

Today we will first use DP for longest common subsequence and then study use two different state spaces: subtrees of a rooted tree and prefixes paired with a target value. We will then preview approximation algorithms by turning the knapsack DP into an FPTAS.

3 Longest common subsequence

Input. Sequences

$$A = a_1 a_2 \cdots a_n, \qquad B = b_1 b_2 \cdots b_m.$$

A *subsequence* is obtained by deleting zero or more elements without changing the order of those retained. Unlike a substring, its elements need not occupy consecutive positions.

Goal. Find the maximum length of a sequence that is a subsequence of both A and B.

Example. If $A =$ `ABCDEFG` and $B =$ `BCDGKAB`, then `BCDG` is a common subsequence of length 4.

Definition (prefix subproblem). For $0 \le i \le n$ and $0 \le j \le m$, let $D[i, j]$ be the length of a longest common subsequence of $a_1 \cdots a_i$ and $b_1 \cdots b_j$.

The empty prefix gives the base cases

$$D[0, j] = D[i, 0] = 0.$$

For $i, j \ge 1$,

$$D[i, j] = \begin{cases} D[i-1, j-1] + 1, & a_i = b_j, \\ \max\{D[i-1, j], D[i, j-1]\}, & a_i \neq b_j. \end{cases}$$

Why the recurrence works. If $a_i = b_j$, some longest common subsequence can end with this common symbol; what precedes it is an LCS of the two shorter prefixes. If $a_i \neq b_j$, a common subsequence cannot use both final symbols, so at least one must be discarded.

The table has $(n+1)(m+1)$ entries, each filled in constant time. Thus the running time is $\mathcal{O}(nm)$ and the straightforward space usage is $\mathcal{O}(nm)$.

3.1 Recovering an actual LCS

Start at (n, m) and trace backward:

- if $a_i = b_j$, output that symbol and move to $(i-1, j-1)$;
- otherwise move to a neighboring state whose value equals $D[i, j]$.

The symbols are discovered in reverse order. The traceback takes $\mathcal{O}(n+m)$ time once the table is available.

Exercise. Draw the DP table for the two strings in the example and recover one LCS. Where can ties occur, and what do they mean?

DP takeaway. The hardest part is usually not filling the table; it is choosing a state that contains exactly the information needed by the recurrence. Next time we will practice this idea on trees and on the knapsack problem.

4 Maximum independent set in a tree

Let $T = (V, E)$ be an undirected tree. An *independent set* is a subset $S \subseteq V$ such that no edge has both endpoints in S.

Goal. Find an independent set of maximum cardinality.

Root the tree at an arbitrary vertex r. For a vertex v, let T_v denote the subtree rooted at v.

Definition (subproblem). Let $M[v]$ be the size of a largest independent set in T_v.

There are two possibilities at v:

- **include v:** then none of its children may be included, but each grandchild roots an independent remaining subproblem;
- **exclude v:** then each child roots an independent subproblem.

Thus

$$M[v] = \max \left\{ 1 + \sum_{u \in \text{grandchildren}(v)} M[u], \sum_{u \in \text{children}(v)} M[u] \right\}.$$

For a leaf v, this gives $M[v] = 1$.

Evaluation order. Run a postorder traversal, so every child and grandchild value is available before computing $M[v]$. Each vertex appears in the computation of only its parent and grandparent, so the total running time is $\mathcal{O}(|V|)$.

Exercise. Draw a rooted tree with at least eight vertices, compute every $M[v]$ bottom-up, and recover an optimal set. What decision must be recorded at each vertex?

4.1 An equivalent two-state formulation

Sometimes it is cleaner to explicitly record the decision at the root of each subtree. Define

$$I[v] = \text{best size in } T_v \text{ subject to including } v,$$
$$X[v] = \text{best size in } T_v \text{ subject to excluding } v.$$

Then

$$I[v] = 1 + \sum_{u \in \text{children}(v)} X[u],$$
$$X[v] = \sum_{u \in \text{children}(v)} \max\{I[u], X[u]\},$$

and the answer is $\max\{I[r], X[r]\}$.

Why this is useful. The state says exactly what constraint the parent needs to impose on a child. This pattern generalizes to many tree problems: create one state for each possible interaction across the edge to the parent.

Exercise. Prove that the one-state and two-state recurrences return the same optimum. Which formulation makes solution recovery easier?

5 The knapsack problem

There are n items. Item i has a positive integer size s_i and a positive integer value v_i. A knapsack has integer capacity B.

Goal. Choose $S \subseteq \{1, \ldots, n\}$ maximizing

$$\sum_{i \in S} v_i \qquad \text{subject to} \qquad \sum_{i \in S} s_i \leq B.$$

Example. Suppose the sizes are $(6, 3, 4, 2)$, the values are $(30, 14, 16, 9)$, and $B = 10$. Items 1 and 3 fit exactly and have total value 46.

Knapsack is NP-hard, so we do not expect an algorithm polynomial in the ordinary bit length of every input unless $\mathsf{P} = \mathsf{NP}$. We can, however, solve it by a dynamic program whose time is polynomial in the *numeric values* of the input.

5.1 A value-indexed dynamic program

Let

$$V_{\text{sum}} = \sum_{i=1}^{n} v_i.$$

Definition (subproblem). For $0 \le i \le n$ and $0 \le v \le V_{\text{sum}}$, let $D[i, v]$ be the minimum total size of a subset of the first i items having total value exactly v. If no such subset exists, set $D[i, v] = +\infty$.

The base cases are

$$D[0, 0] = 0, \qquad D[0, v] = +\infty \quad (v > 0).$$

For $i \ge 1$,

$$D[i, v] = \begin{cases} \min\{D[i-1, v],\ s_i + D[i-1, v - v_i]\}, & v \ge v_i, \\ D[i-1, v], & v < v_i. \end{cases}$$

The answer is the largest v for which $D[n, v] \le B$.

Correctness. Any subset of the first i items either excludes item i, giving the first candidate, or includes item i, leaving value $v - v_i$ to be realized by the first $i - 1$ items. The recurrence takes the smaller feasible size.

The table has $\mathcal{O}(nV_{\text{sum}})$ entries and constant work per entry. Since $V_{\text{sum}} \le n \max_i v_i$, the running time is

$$\mathcal{O}(nV_{\text{sum}}) = \mathcal{O}(n^2 \max_i v_i).$$

5.2 Why this is pseudo-polynomial

Writing an integer x in binary takes $\Theta(\log x)$ bits. The input length contains terms such as $\log v_i$ and $\log B$, but the DP time depends on the values themselves. For example, $v_i = 2^k$ needs only $k + 1$ bits to write, while an iteration for every value up to v_i takes exponential time in k.

An algorithm polynomial in the numeric magnitudes of integer inputs, but not necessarily in their encoding length, is called *pseudo-polynomial*.

Exercise. Fill the value-indexed table for the example and recover the chosen items. Then design the familiar capacity-indexed recurrence and compare its running time.

6 Preview: an FPTAS for knapsack

The pseudo-polynomial DP is slow only because its value axis can be enormous. The natural idea is to round values so that the table becomes small. Rounding must be coarse enough to improve the running time but fine enough to control the total loss.

Approximation language. For a maximization problem, a $(1-\varepsilon)$-approximation returns a feasible solution of value at least $(1-\varepsilon)\,\mathrm{OPT}$, where $0 < \varepsilon < 1$. A polynomial-time approximation scheme (PTAS) has polynomial running time for every fixed ε. A fully polynomial-time approximation scheme (FPTAS) has running time polynomial in both the input length and $1/\varepsilon$.

First discard items with $s_i > B$, since they can never be selected. If no item remains, return the empty set. Otherwise let

$$V = \max_i v_i.$$

Because an item attaining V is feasible by itself, $V \le \mathrm{OPT}$.

Choose the scaling unit

$$K = \frac{\varepsilon V}{n}$$

and replace each value by the nonnegative integer

$$\widehat{v}_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

Run the value-indexed knapsack DP using the scaled values $\widehat{v}_i$, and let $\widehat{S}$ be the feasible subset with maximum total scaled value.

Theorem (knapsack FPTAS). For every $0 < \varepsilon < 1$, the scaling algorithm returns a feasible set $\widehat{S}$ such that

$$v(\widehat{S}) \ge (1-\varepsilon)\,\mathrm{OPT}\,.$$

Its running time is $\mathcal{O}(n^3/\varepsilon)$ under the same unit-cost arithmetic convention as the pseudo-polynomial DP.

Running time. For every item,

$$\widehat{v}_i \le \frac{v_i}{K} \le \frac{V}{K} = \frac{n}{\varepsilon}.$$

Hence

$$\sum_{i=1}^{n} \widehat{v}_i \le \frac{n^2}{\varepsilon},$$

and the value-indexed DP takes

$$\mathcal{O}\left(n \sum_i \widehat{v}_i\right) = \mathcal{O}(n^3/\varepsilon)$$

time. This is polynomial in both the encoded input length and $1/\varepsilon$.

6.1 Approximation guarantee

Rounding down loses less than K on each selected item:

$$v_i - K < K\widehat{v}_i \leq v_i.$$

Let S^* be an optimal solution. Because $\widehat{S}$ maximizes scaled value among all feasible sets,

$$\begin{aligned} v(\widehat{S}) &\geq K \sum_{i \in \widehat{S}} \widehat{v}_i \\ &\geq K \sum_{i \in S^*} \widehat{v}_i \\ &\geq \sum_{i \in S^*} (v_i - K) \\ &\geq \mathrm{OPT} - nK \\ &= \mathrm{OPT} - \varepsilon V \\ &\geq (1 - \varepsilon)\,\mathrm{OPT}\,. \end{aligned}$$

Thus the total rounding loss is at most $nK = \varepsilon V$, and $V \leq \mathrm{OPT}$ converts this additive loss into the desired multiplicative guarantee.

What to notice. The approximation algorithm is still the same dynamic program. We changed the numbers used to index its states, trading a controlled amount of objective value for a polynomial number of states. Later, in the approximation-algorithms unit, we will develop this design pattern and its analysis more systematically.

Exercise. For the running example, take $\varepsilon = 1/2$. Compute K and every scaled value $\widehat{v}_i$, run the scaled DP, and compare its returned value with $(1 - \varepsilon)\,\mathrm{OPT}$. Is the proven bound tight on this input?

Takeaway. DP can make an exponential search manageable by merging partial solutions that have the same relevant state. When the exact state space is too large, rounding can compress it while preserving a provable approximation. Next time we continue DP with uncertainty and finite-horizon decision making.

CS 6515: Graduate Algorithms

Lecture 5: Knapsack FPTAS and Finite-Horizon MDPs

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW2 is out; due Sep 17. Although slightly long (6 pbs), you don't have to attempt them all.
2. Exam 1 is Sep 22.

2 Where we left off: value-indexed knapsack

There are n items, with positive integer sizes s_i and values v_i, and the knapsack has capacity B. Let

$$V_{\text{sum}} = \sum_{i=1}^{n} v_i.$$

Recall the value-indexed subproblem

$$D[i, v] = \text{minimum total size of a subset of the first } i \text{ items having value exactly } v,$$

where an impossible state has value $+\infty$. The recurrence is

$$D[i, v] = \begin{cases} \min\{D[i-1, v],\ s_i + D[i-1, v-v_i]\}, & v \geq v_i, \\ D[i-1, v], & v < v_i. \end{cases}$$

The optimum is the largest v for which $D[n, v] \leq B$. The resulting exact algorithm runs in

$$\mathcal{O}(nV_{\text{sum}}) = \mathcal{O}(n^2 \max_i v_i)$$

time. This is *pseudo-polynomial*: it is polynomial in the numerical values v_i, but not necessarily in their binary encoding lengths.

The obstacle. The dynamic program is slow only because its value axis can be enormous. Can we compress that axis while losing only a small amount of objective value?

3 An FPTAS for knapsack

For a maximization problem, a $(1-\varepsilon)$-approximation returns a feasible solution of value at least $(1-\varepsilon)\,\text{OPT}$, where $0 < \varepsilon < 1$. A polynomial-time approximation scheme (PTAS) has polynomial running time for every fixed ε. A fully polynomial-time approximation scheme (FPTAS) has running time polynomial in both the input length and $1/\varepsilon$.

First discard items with $s_i > B$, since they can never be selected. If no item remains, return the empty set. Otherwise let

$$V_{\max} = \max_i v_i.$$

An item attaining $V_{\max}$ is feasible by itself, so $V_{\max} \le \mathrm{OPT}$.

Choose the scaling unit

$$K = \frac{\varepsilon V_{\max}}{n}$$

and replace each value by the nonnegative integer

$$\widehat{v}_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

Run the value-indexed knapsack DP using the scaled values $\widehat{v}_i$, and let $\widehat{S}$ be the feasible subset with maximum total scaled value.

Theorem (knapsack FPTAS). For every $0 < \varepsilon < 1$, the scaling algorithm returns a feasible set $\widehat{S}$ such that

$$v(\widehat{S}) \ge (1-\varepsilon)\,\mathrm{OPT}\,.$$

Its running time is $\mathcal{O}(n^3/\varepsilon)$ under the unit-cost arithmetic convention used for the pseudo-polynomial DP.

Running time. For every remaining item,

$$\widehat{v}_i \le \frac{v_i}{K} \le \frac{V_{\max}}{K} = \frac{n}{\varepsilon}.$$

Consequently,

$$\sum_i \widehat{v}_i \le \frac{n^2}{\varepsilon}, \qquad \mathcal{O}\left(n \sum_i \widehat{v}_i\right) = \mathcal{O}(n^3/\varepsilon).$$

Thus the scaled table has polynomial size in both the input length and $1/\varepsilon$.

3.1 Approximation guarantee

Rounding down loses less than K on each item:

$$v_i - K < K\widehat{v}_i \le v_i.$$

Let S^* be an optimal solution. Since $\widehat{S}$ maximizes scaled value among all feasible sets,

$$\begin{aligned} v(\widehat{S}) &\ge K \sum_{i \in \widehat{S}} \widehat{v}_i \\ &\ge K \sum_{i \in S^*} \widehat{v}_i \\ &> \sum_{i \in S^*} (v_i - K) \\ &\ge \mathrm{OPT} - nK \\ &= \mathrm{OPT} - \varepsilon V_{\max} \\ &\ge (1-\varepsilon)\,\mathrm{OPT}\,. \end{aligned}$$

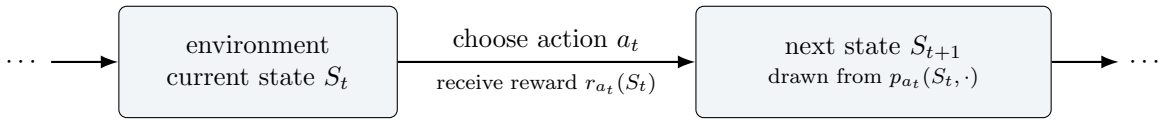
The first inequality translates scaled value back to true value; the second uses optimality for the scaled instance; and the remaining inequalities bound the accumulated rounding loss.

What to notice. The approximation algorithm uses the same dynamic program. We changed only the numbers indexing its states, trading an additive loss of at most nK for a polynomial number of states.

Exercise. For sizes $(6, 3, 4, 2)$, values $(30, 14, 16, 9)$, capacity $B = 10$, and $\varepsilon = 1/2$, compute K and the scaled values. Compare the returned solution with $(1 - \varepsilon)\,\mathrm{OPT}$. Is the guarantee tight on this instance?

4 Finite-horizon Markov decision processes

From deterministic DP to decisions under uncertainty. In knapsack, choosing whether to include an item determined the next subproblem. We now consider decisions whose outcomes are uncertain: an action leads to a *distribution* over possible next states. The dynamic-programming principle remains the same—summarize the future by the value of a smaller state.



Markov decision process. A finite-horizon Markov decision process (MDP) consists of:

- a finite state space $\mathcal{S}$;
- a finite action space $\mathcal{A}$;
- a starting state $s_1 \in \mathcal{S}$;
- transition probabilities $p_a(s, s')$ for every a, s, s', where

$$p_a(s, s') \geq 0, \qquad \sum_{s' \in \mathcal{S}} p_a(s, s') = 1;$$

- an immediate reward $r_a(s) \in \mathbb{R}$ for taking action a in state s;
- a time horizon T.

When action a_t is taken in state S_t, reward $r_{a_t}(S_t)$ is received, and the next state S_{t+1} is drawn according to $p_{a_t}(S_t, \cdot)$.

Definition (policy). A policy specifies which action to take at each step. In full generality, it could depend on the entire history of visited states and chosen actions.

The objective is

$$\max_{\pi} \ \mathbb{E}\left[\sum_{t=1}^{T} r_{a_t}(S_t)\right].$$

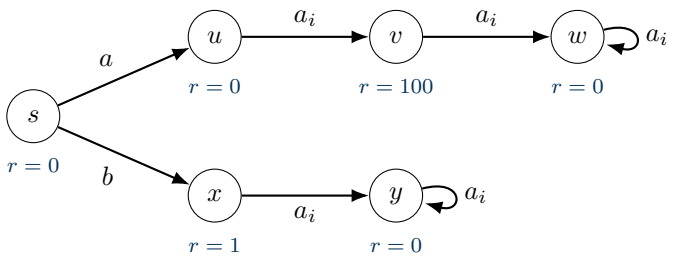
The transition table has $\Theta(|\mathcal{A}||\mathcal{S}|^2)$ entries when it is stored explicitly.

5 Two small examples

5.1 Changing the horizon can change the first action

Imagine a start state with two actions. Action a enters a two-step route that eventually pays reward 100, while action b enters a one-step route that pays reward 1. All transitions are deterministic.

- With one step remaining, neither route may have time to pay.
- With two steps remaining, the short route can be better.
- With three steps remaining, the long route can be much better.

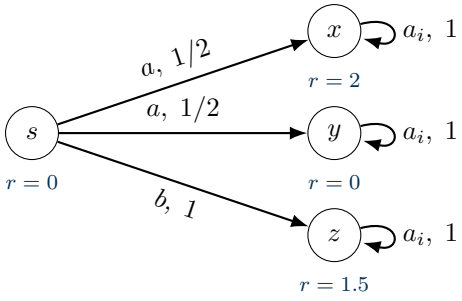


The label below a state is its immediate reward; a_i denotes either available action.

The optimal action can therefore depend on both the current state and the number of steps remaining.

5.2 Expected reward, not best possible reward

From a state s, suppose action a moves with probability $1/2$ to a state paying 2 next step and with probability $1/2$ to a state paying 0. Action b moves deterministically to a state paying 1.5.



With two steps remaining, the continuation values are

$$\frac{1}{2}(2) + \frac{1}{2}(0) = 1 \qquad \text{and} \qquad 1.5,$$

so b is better even though a has the better possible outcome.

Exercise. For the first example, determine an optimal first action and the resulting total reward for horizons $T = 1, 2, 3$. For the second example, do the same for horizons $T = 1, 2$. What distinct lessons do the two examples illustrate?

6 The Bellman recurrence

Definition (optimal value). Let $V^*(s, t)$ be the maximum expected total reward obtainable when the current state is s and exactly t actions remain.

The base case is

$$V^*(s, 0) = 0 \qquad \text{for every } s \in \mathcal{S}.$$

For $t \geq 1$, conditioning on the first action and next state gives

$$V^*(s, t) = \max_{a \in \mathcal{A}} \left\{ r_a(s) + \sum_{s' \in \mathcal{S}} p_a(s, s') V^*(s', t-1) \right\}.$$

This is the *Bellman optimality equation.*

Observation. There is an optimal policy whose action depends only on the current state and the number of steps remaining. The earlier history contains no additional information needed by the Bellman recurrence.

Observation. There is an optimal deterministic policy. At each state and time, choose an action attaining the maximum in the Bellman equation. Randomizing among actions cannot improve on the best action's expected value.

6.1 Bottom-up algorithm

1. Set $V[s, 0] = 0$ for every state s.
2. For $t = 1, 2, \ldots, T$ and every s, evaluate the Bellman expression for each action a.
3. Store both $V[s, t]$ and an action $\text{act}[s, t]$ attaining the maximum.
4. Starting in s_1 with T steps remaining, use $\text{act}[s, t]$ and decrement t after every transition.

Theorem (finite-horizon MDP). An optimal finite-horizon policy can be computed in

$$\mathcal{O}\big(T|\mathcal{A}||\mathcal{S}|^2\big)$$

time and $\mathcal{O}(T|\mathcal{S}|)$ space when transitions are represented explicitly.

Running time. There are $T|\mathcal{S}|$ DP states. For each, we consider $|\mathcal{A}|$ actions and sum over $|\mathcal{S}|$ possible next states.

Correctness. Induct on t. The base case is immediate. For the inductive step, every policy chooses some first action a. Conditional on the next state s', the inductive hypothesis says that no continuation beats $V^*(s', t-1)$. Choosing the maximizing first action and then an optimal continuation attains the upper bound.

Exercise. If only the optimal expected value is required, reduce the space to $\mathcal{O}(|\mathcal{S}|)$. Why does recovering the entire policy require storing more information or recomputing values?

CS 6515: Graduate Algorithms

Lecture 6: Graph Representations, BFS, Shortest Paths, and MSTs

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

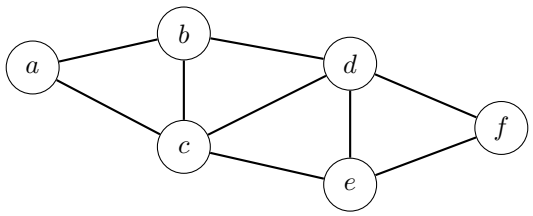
From the previous lecture. Dynamic programs can be viewed as dependency graphs whose vertices are subproblems. An MDP also has a transition graph, although an action may lead to a distribution over next states. We now make the graph itself the input and ask structural questions about reachability, shortest paths, and cheap ways to connect all vertices.

1 Announcements

1. HW1 graded. Submit any regrade requests very soon.
2. HW2 is due Sep 17.
3. Exam 1 is Sep 22.

2 Graphs and their representations

A graph is $G = (V, E)$, where V is a set of vertices and E is a set of edges. Write $n = |V|$ and $m = |E|$. Graphs may be directed or undirected, weighted or unweighted.



An undirected graph with $n = 6$ and $m = 9$. For example, $\deg(c) = 4$ and the neighbors of c are a, b, d, e.

For an undirected graph, the degree $\deg(v)$ is the number of edges incident to v, and

$$\sum_{v \in V} \deg(v) = 2m.$$

Representation	Space	Typical operation
Adjacency matrix	$\Theta(n^2)$	Test $(u, v) \in E$ in $\mathcal{O}(1)$
Adjacency lists	$\Theta(n + m)$	Scan all neighbors in $\Theta(\deg(u))$

Adjacency lists are the default for sparse-graph algorithms because scanning all lists costs $\Theta(n + m)$.

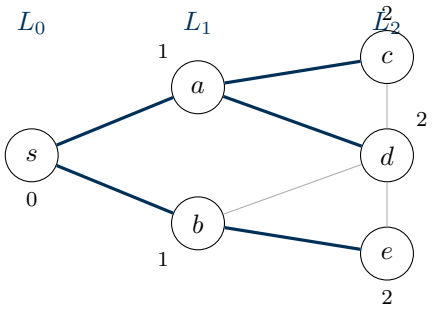
3 Breadth-first search

In an unweighted graph, the length of a path is its number of edges. Given a source s, breadth-first search (BFS) explores vertices in nondecreasing order of their shortest-path distance from s.

Algorithm. Initialize $\text{dist}[s] = 0$, set all other distances to $+\infty$, and place s in a first-in, first-out queue. Repeatedly remove a vertex u from the queue. For every neighbor v that has not been discovered, set

$$\text{dist}[v] = \text{dist}[u] + 1, \qquad \text{parent}[v] = u,$$

and append v to the queue.



BFS processes the distance layers L_0, L_1, L_2 in order. The blue edges are one possible parent tree; the gray edges do not shorten any layer.

Theorem (BFS correctness). When BFS finishes, $\text{dist}[v]$ is the minimum number of edges on an s-v path for every reachable vertex v. Parent pointers form a shortest-path tree.

Proof idea. The queue processes vertices one distance layer at a time. When v is first discovered from u, BFS has already found a path of length $\text{dist}[u] + 1$. If a shorter path to v existed, its preceding vertex would have belonged to an earlier layer and would have discovered v first.

Each vertex enters the queue once and each adjacency-list entry is scanned once, so BFS takes $\mathcal{O}(n + m)$ time.

Exercise. Run BFS from a chosen source on a graph containing a cycle. Label distance layers, draw the parent tree, and explain why non-tree edges do not invalidate the distances.

4 Nonnegative weighted shortest paths

BFS works because every edge contributes the same unit length. Now every edge e has a length $w(e) \geq 0$, and path length is the sum of its edge lengths.

4.1 A pseudo-polynomial reduction

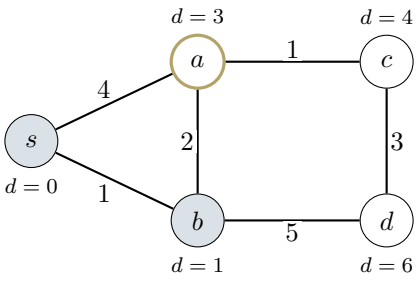
If all weights are positive integers, replace an edge of weight w by a path of w unit edges, introducing $w - 1$ new vertices. BFS in the expanded graph finds weighted shortest paths, but the graph size can depend on the numeric weights rather than their bit lengths. This is therefore only a pseudo-polynomial approach.



4.2 Dijkstra's algorithm

Dijkstra's algorithm maintains a tentative distance $d[v]$ and repeatedly settles the unsettled vertex of minimum tentative distance.

1. Set $d[s] = 0$ and $d[v] = +\infty$ for $v \neq s$. Store unsettled vertices in a min-priority queue keyed by $d[v]$.
2. Extract a vertex u of minimum key and mark it settled.
3. For every outgoing edge (u, v) with v unsettled, *relax* it: if
$$d[u] + w(u, v) < d[v],$$
update $d[v]$ and set parent$[v] = u$.
4. Repeat until the priority queue is empty.



After settling s and b, relaxing edges out of b gives $d[a] = 3$ and $d[d] = 6$. Vertex a is the next minimum-key vertex.

Theorem (Dijkstra correctness). If all edge weights are nonnegative, then whenever a vertex u is extracted, $d[u]$ equals the true shortest-path distance from s to u.

Proof idea. Suppose u is the first incorrectly settled vertex and consider a shortest path from s to u. Let (x, y) be the first edge on that path leaving the settled set. Vertex x was settled correctly, so relaxing (x, y) made $d[y]$ at most the length of the path prefix to y. Nonnegative weights imply that this prefix is no longer than the full path to u. Since u had minimum key, $d[u] \le d[y] \le \text{dist}(s, u)$. Every finite tentative distance is the length of an actual s–u path, so $d[u] \ge \text{dist}(s, u)$ as well. Hence $d[u] = \text{dist}(s, u)$.

With a binary heap, there are n extract-min operations and at most m decrease-key operations, for a running time of

$$\mathcal{O}((n + m) \log n).$$

4.3 Remarks: beyond the binary-heap bound

Fibonacci heaps. Dijkstra performs at most m decrease-key operations but only n extract-min operations. A Fibonacci heap supports decrease-key in $\mathcal{O}(1)$ amortized time and extract-min in $\mathcal{O}(\log n)$ amortized time. Thus, it improves the running time to

$$\mathcal{O}(m + n \log n).$$

This classic theoretical improvement is due to Fredman and Tarjan (1987); binary heaps are often simpler and faster in practice.

This is still an active area of research. For directed graphs with real nonnegative edge weights, a 2025 result of Duan, Mao, Mao, Shu, and Yin broke the long-standing "sorting barrier" with a deterministic

$$\mathcal{O}\Big(m \log^{2/3} n\Big)$$

algorithm in the comparison-addition model (paper).

Dijkstra's algorithm itself does not allow negative edges. Nevertheless, for directed graphs with integral, possibly negative edge weights and no reachable negative cycle, recent randomized algorithms have achieved near-linear time. A 2022 breakthrough gave $\mathcal{O}(m \log^8 n \log W)$ time, and a 2023 improvement reduced this to

$$\mathcal{O}\big(m \log^2 n \, \log(nW) \, \log\log n\big)\,,$$

where W bounds the magnitude of the most negative edge (2022 paper; 2023 improvement). These algorithms use ideas substantially beyond Dijkstra's algorithm.

Exercise. Give a small graph with a negative edge on which Dijkstra's algorithm can settle an incorrect distance. Which line of the proof uses nonnegativity?

5 Minimum spanning trees

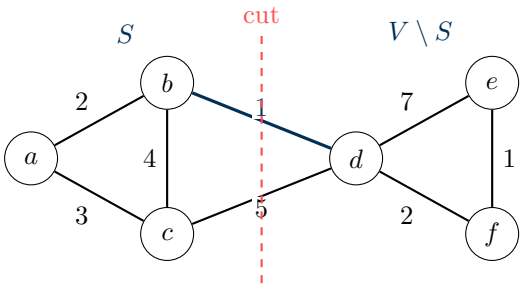
Shortest paths optimize routes from a source. A minimum spanning tree instead asks for one inexpensive backbone connecting all vertices. Let $G = (V, E)$ be a connected, undirected graph with nonnegative edge weights $w : E \to \mathbb{R}_{\ge 0}$.

Goal. Find $T \subseteq E$ that connects all vertices and minimizes

$$w(T) = \sum_{e \in T} w(e).$$

Observation. Some optimal solution is a tree. If a connected solution contains a cycle, remove a maximum-weight edge on that cycle; connectivity is preserved and the cost does not increase.

For $\varnothing \neq S \subsetneq V$, the cut $(S, V \setminus S)$ consists of all edges with one endpoint in S and the other outside S.



For the displayed cut, (b, d) is the lightest crossing edge. The cut property says that some MST contains it.

Theorem (cut property). Let e be a minimum-weight edge crossing any cut $(S, V \setminus S)$. There is a minimum spanning tree containing e. If e is the unique lightest crossing edge, every minimum spanning tree contains e.

Proof. Let T be an MST. If $e \in T$, we are done. Otherwise, adding e to T creates a cycle. Because the cycle crosses from S to $V \setminus S$ using e, it must cross back on some edge $f \in T$. By the choice of e, $w(e) \leq w(f)$. Thus

$$T' = T \cup \{e\} \setminus \{f\}$$

is a spanning tree with $w(T') \leq w(T)$. Since T was optimal, T' is an MST containing e. With a strict inequality against every other crossing edge, an MST omitting e would be improved, which is impossible. □

Exercise. Draw a weighted connected graph and three different cuts. Mark a light edge for each cut. Can two different light edges be certified simultaneously by the theorem?

Next time. The cut property immediately yields Prim's minimum-spanning-tree algorithm. We will then study matchings and their augmenting paths.

CS 6515: Graduate Algorithms

Lecture 7: Prim's Algorithm and Matchings

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. HW2 is due Sep 17.
2. Exam 1 is Sep 22. Syllabus Lectures 1-8.

2 Recap: the MST cut property

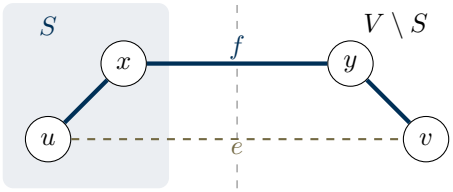
Let $G = (V, E)$ be a connected, undirected graph with edge weights $w(e)$. Write $n = |V|$ and $m = |E|$. A *spanning tree* connects all vertices without a cycle, so it has exactly $n - 1$ edges. Our goal is to find a spanning tree T minimizing $w(T) = \sum_{e \in T} w(e)$.

With nonnegative weights, some cheapest connected spanning subgraph is a tree: repeatedly delete a cycle edge without increasing cost. A minimum spanning tree need not give shortest routes from a root; its objective is the *total edge weight*.

Definition (cut). For $\varnothing \neq S \subsetneq V$, the cut $(S, V \setminus S)$ consists of all edges with one endpoint in S and the other outside S. A light edge of this cut is an edge of minimum weight among its crossing edges.

Theorem (cut property). For any nontrivial cut, any light edge e belongs to *some* MST. If e is the unique light edge of that cut, it belongs to *every* MST.

Proof plan. Start with an MST that omits e. Adding e creates a cycle; deleting an appropriate edge from that cycle restores a tree at no greater cost.



Schematic: the solid path is in T^*; adding the dashed edge e closes a cycle. The path must cross the cut on another edge f.

Exchange proof. Let T^* be an MST. If $e \in T^*$, we are done. Otherwise, $T^* \cup \{e\}$ contains one cycle. Since e crosses the cut, the rest of the cycle must cross back on an edge $f \in T^*$. Because e is light, $w(e) \leq w(f)$. Consequently,

$$T' = T^* \cup \{e\} \setminus \{f\}, \qquad w(T') = w(T^*) + w(e) - w(f) \leq w(T^*).$$

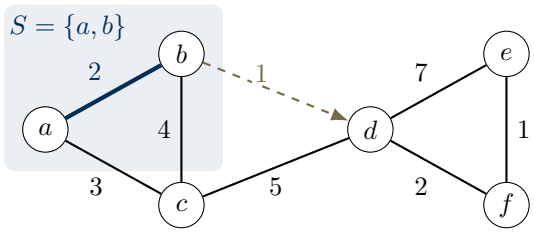
Deleting a cycle edge preserves connectivity, so T' is a spanning tree. Optimality of T^* implies that T' is also an MST, now containing e. If e is uniquely light, the inequality would be strict; hence no MST can omit it. □

3 Prim's algorithm

Let G be connected and undirected. Prim's algorithm grows one tree from a chosen root r.

Ties and compatible choices. In an equal-weight triangle, each edge is light for a cut but no tree contains all three. Thus "some MST" cannot be replaced by "every MST" when there are ties. Prim chooses cuts whose light edges are compatible with all earlier choices.

1. Start with the visited set $S = \{r\}$ and $T = \varnothing$.
2. Choose a minimum-weight edge (u, v) crossing $(S, V \setminus S)$, with $u \in S$ and $v \notin S$.
3. Add (u, v) to T, add v to S, and repeat until $S = V$.



Starting at a, Prim first selects (a, b). The crossing edges now have weights $3, 4, 1$, so the next edge is (b, d) (dashed).

Proof plan and invariant. We must show that *some MST contains all edges selected so far*, not just that each selected edge belongs to some MST on its own.

Correctness. Initially $T = \varnothing$, so the invariant holds. Suppose an MST T^* contains T, and let e be the next minimum-weight edge across $(S, V \setminus S)$. If $e \notin T^*$, adding it creates a cycle. That cycle has another crossing edge f, with $w(f) \geq w(e)$. No edge of T crosses this cut, so $f \notin T$. Replacing f by e gives an MST containing $T \cup \{e\}$. Thus the invariant is preserved. At the end T has $n - 1$ edges, so it is itself that MST. □

3.1 Priority-queue implementation

For every unvisited vertex v, maintain

$$\text{key}[v] = \min\{w(u, v) : u \in S,\ (u, v) \in E\},$$

where the minimum of an empty set is $+\infty$. Store a parent attaining any finite minimum. Initialize these keys by scanning the edges out of r. A min-priority queue returns the unvisited vertex with the smallest key. When a vertex u is added, scan its incident edges and decrease the keys of its unvisited neighbors when possible.

With adjacency lists and a binary heap, Prim's algorithm takes

$$\mathcal{O}((n + m) \log n) = \mathcal{O}(m \log n)$$

time for a connected graph.

As with Dijkstra, there are at most n extract-min operations and m decrease-key operations. Fibonacci heaps give $\mathcal{O}(m + n \log n)$ time.

Compare with Dijkstra. The queue operations look the same, but the keys mean different things:

	Meaning of key at v	Candidate when scanning (u, v)
Dijkstra	Length of a path from s	$d[u] + w(u, v)$
Prim	Cost of one edge from S	$w(u, v)$

An MST need not be a shortest-path tree. Prim's exchange proof also works with negative edge weights, unlike Dijkstra's proof. Here an MST always means a minimum-weight *spanning tree*.

Exercise. Continue Prim's algorithm on the displayed graph. Record the visited set, keys, and selected edge after every iteration. Can different tie-breaking choices produce different MSTs?

4 Matchings

Let $G = (V, E)$ be an undirected graph.

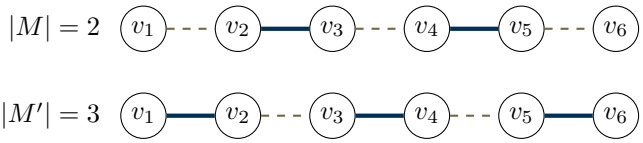
Definition (matching). A matching is a set $M \subseteq E$ such that no vertex is incident to more than one edge of M.

Goal. Find a matching of maximum cardinality.

A matching is *maximal* if no edge can be added to it, while it is *maximum* if no larger matching exists. These words are not equivalent: a greedy matching can be maximal but smaller than a maximum matching.

Matching models one-to-one assignments, including courses to classrooms, donors to recipients, and participants on two sides of a marketplace.

Proof plan. Adding one edge may be impossible because its endpoints are already matched. Instead, look for a chain of reassignments joining two unmatched endpoints and gaining one edge overall. We will prove that if no such improvement exists, the matching is maximum.



Solid blue edges form the current matching; dashed edges are outside it. The upper matching is maximal but not maximum. Flipping all five path edges gives the lower matching.

Definition (augmenting path). Given a matching M, an augmenting path is a simple path whose edges alternate between $E \setminus M$ and M, and whose two endpoints are unmatched by M.

An augmenting path begins and ends with unmatched edges. Therefore it contains one more unmatched edge than matched edge.

Observation. If P is an augmenting path, then

$$M \triangle E(P) = \big(M \setminus E(P)\big) \cup \big(E(P) \setminus M\big)$$

is a matching of size $|M| + 1$. We say that we *augment along* P.

Every internal vertex of P loses one matched edge and gains one; each endpoint gains its first matched edge. All other vertices are unchanged. This proves feasibility as well as the increase in size.

Exercise. In the displayed example, why can no single edge be added to the upper matching? Why does flipping the entire path still improve it?

5 When is a matching maximum?

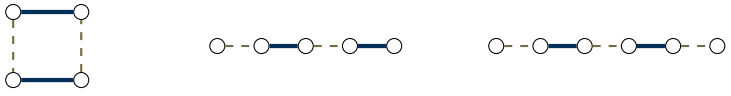
Theorem (Berge's augmenting-path theorem). A matching M is maximum if and only if there is no augmenting path with respect to M.

One direction follows from the preceding observation: an augmenting path proves that M is not maximum.

For the converse, suppose a larger matching M^* exists. Consider the symmetric difference

$$M \triangle M^*.$$

Every vertex has degree at most two in this subgraph, so every component with an edge is a path or a cycle. Along each component, edges alternate between M and M^*.



cycle
2 from each

even path
2 from each

augmenting path for M
3 from M^*, 2 from M

Components of $M \triangle M^*$: solid blue denotes $M \setminus M^*$; dashed gold denotes $M^* \setminus M$. An odd path can instead favor M; the total size difference guarantees that some path favors M^*.

An alternating cycle contains equally many edges from the two matchings. So does an even alternating path. Since $|M^*| > |M|$, some component must contain one more M^* edge than M edge. That component is a path beginning and ending with M^* edges; its endpoints are unmatched by M. It is therefore an augmenting path for M, a contradiction. □

To justify the endpoint claim: an endpoint cannot have another M edge in the symmetric difference, or it would not be an endpoint. It cannot have an edge common to M and M^* either, because it already has an incident M^* edge in this path.

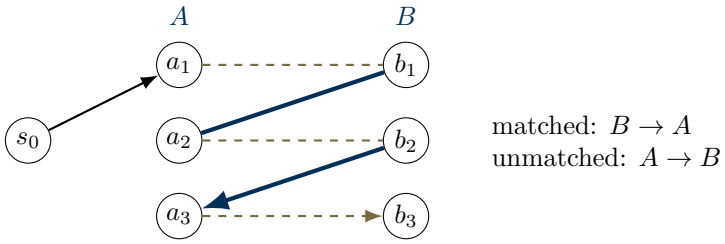
Algorithmic plan. Start with $M = \varnothing$. Repeatedly find an augmenting path and augment. If none exists, the theorem certifies that M is maximum. The remaining issue is how to find such a path efficiently.

General graphs. Berge's theorem applies to all undirected graphs. Finding an augmenting path in a general graph is more subtle because of odd cycles; Edmonds' blossom algorithm handles this case in polynomial time. We focus on the simpler bipartite case.

6 Bipartite graphs

Definition (bipartite graph). A graph is bipartite if $V = A \cup B$, $A \cap B = \varnothing$, and every edge has one endpoint in A and the other in B.

For a current matching M, orient every unmatched edge from A to B and every matched edge from B to A. Add a super-source connected to unmatched vertices of A, and search for a directed path to an unmatched vertex of B. Such a path is exactly an augmenting path.



Here $M = \{(a_2, b_1), (a_3, b_2)\}$. Search from the only unmatched left vertex a_1 reaches the unmatched right vertex b_3 via $a_1, b_1, a_2, b_2, a_3, b_3$. Flipping gives a matching of size 3.

This is the same six-vertex path as before, drawn with its bipartition visible. BFS from Lecture 6 supplies the search; parent pointers recover the actual path. After flipping, reverse the orientations of its edges and update which vertices are unmatched before the next search.

A BFS or DFS finds one in $\mathcal{O}(n+m)$ time. Since each augmentation increases the matching size by one and a matching has at most $n/2$ edges, this yields a simple polynomial-time algorithm using

$$\mathcal{O}(n(n+m))$$

time. Faster algorithms can augment along several paths at once.

Exercise. Prove the correspondence between directed paths in the oriented graph and augmenting paths. Why does this straightforward orientation argument become more subtle in a nonbipartite graph?

Bridge to max flow. The alternating-edge idea is a special case of residual capacity. Next time we develop maximum flow and then recover bipartite matching as an application.

CS 6515: Graduate Algorithms

Lecture 8: Maximum Flow and Minimum Cut

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

From matching to flow. Last lecture, an augmenting path improved a matching by undoing some old choices and making new ones. Maximum flow generalizes this idea: we route quantities through a directed network, and reverse residual edges let us revise earlier routing decisions. We will again use BFS to find paths, then use a cut to certify that no further improvement is possible.

1 Flow networks

A flow network is a directed graph $G = (V, E)$ with a nonnegative integer capacity c_e on every edge e, a source $s \in V$, and a sink $t \in V$, with $s \neq t$. Write $n = |V|$ and $m = |E|$. Capacities bound how much flow an edge can carry; they are not path lengths. We assume integral capacities for the running-time and termination analysis below; the definition of a flow still permits real values.

Definition (feasible flow). A flow assigns a nonnegative number f_e to every edge and satisfies:

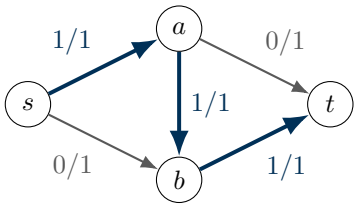
1. **capacity constraints:** $0 \leq f_e \leq c_e$ for all $e \in E$;
2. **flow conservation:** for every $v \in V \setminus \{s, t\}$,

$$\sum_{(u,v)\in E} f_{uv} = \sum_{(v,w)\in E} f_{vw}.$$

The value of a flow is its net flow out of the source:

$$|f| = \sum_{(s,v)\in E} f_{sv} - \sum_{(v,s)\in E} f_{vs}.$$

The maximum-flow problem asks for a feasible flow of maximum value.

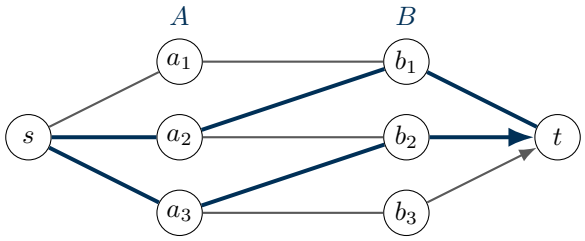


edge labels: f_e/c_e
blue: one unit of flow
$|f| = 1$

Running example. Every capacity is 1. One unit follows s, a, b, t; conservation holds at both a and b. Can we send a second unit without revising this route?

1.1 Bipartite matching as maximum flow

For a bipartite graph with sides A and B, direct every original edge from A to B and give it capacity 1. Add a source edge (s, a) of capacity 1 for each $a \in A$, and a sink edge (b, t) of capacity 1 for each $b \in B$. Every integral flow unit selects one edge between A and B, and capacity 1 ensures no vertex is selected twice. Thus an integral maximum flow corresponds to a maximum matching.



All edges have capacity 1. The dark edges carry one unit each, encoding the Lecture 7 matching $\{(a_2, b_1), (a_3, b_2)\}$ as a flow of value 2.

Why integrality matters. For an integral flow, conservation and the source/sink capacities ensure that the middle edges with flow 1 form a matching. A fractional flow need not select a matching this way. We will show that integer capacities guarantee an *integral optimal flow*, completing the reduction.

Exercise. Prove both directions of the correspondence: turn a matching into a flow, and turn an integral flow into a matching of the same value.

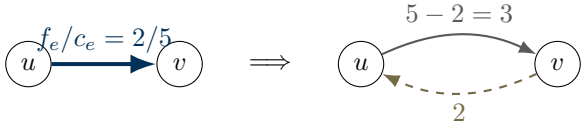
2 Residual graphs

A current flow may use an edge that we later regret. A residual graph records both unused capacity and our ability to cancel previously sent flow.

Definition (residual graph). Given a feasible flow f, the residual graph G_f has vertex set V. For each original edge $e = (u, v)$:

- it has a forward residual edge (u, v) of capacity $c_e - f_e$;
- it has a reverse residual edge (v, u) of capacity f_e.

Only edges of positive residual capacity need to be stored. If residual edges with the same ordered endpoints arise (for example, from original edges in both directions), keep them as separate labeled edges, each linked to its original edge. This makes every update unambiguous.



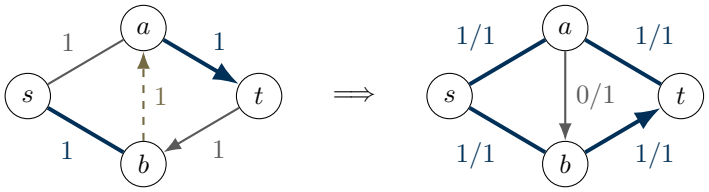
Forward capacity allows 3 more units; reverse capacity allows us to cancel up to 2 units. A reverse edge does not create an original edge.

An s-t path in G_f is an *augmenting path*. Its bottleneck is

$$\Delta = \min\{c_f(e) : e \text{ lies on the path}\}.$$

Augmenting by Δ increases flow on an original edge traversed forward and decreases flow on an original edge traversed backward. The result remains feasible and its value increases by Δ.

Why feasibility is preserved. The bottleneck prevents any original flow from exceeding capacity or falling below zero. Each residual edge changes the net outgoing flow at its tail by $+\Delta$ and at its head by $-\Delta$, whether it adds or cancels flow. These changes cancel at every internal path vertex. Only the source and sink have a net change.



residual graph of the value-1 flow updated flow, $|f| = 2$

In the running example, augment along s, b, a, t with bottleneck 1. The dashed reverse edge $b \to a$ cancels the old unit on $a \to b$. The updated flow uses the two routes s, a, t and s, b, t.

Connection to matching. In the unit-capacity matching network, a used middle edge has only a reverse residual edge, and an unused one has only a forward edge. These are exactly the orientations from Lecture 7. Flow augmentation generalizes flipping matched and unmatched edges.

Exercise. After the first route s, a, b, t in the running example, show that a search using only unused forward capacity would get stuck. Which reverse edge repairs this decision, and why does using it preserve conservation?

3 The Ford-Fulkerson method

1. Start with the zero flow.
2. Construct the residual graph G_f.
3. Find any s-t path P in G_f. If none exists, stop.
4. Augment by the bottleneck capacity of P and return to step 2.

If all capacities are integers, all flow and residual values remain integral. Every augmentation then increases $|f|$ by at least one. A graph search takes $\mathcal{O}(m+n)$ time, so a simple bound is

$$\mathcal{O}\big((m+n)(1+|f^*|)\big),$$

where $|f^*|$ is the maximum-flow value. This is pseudo-polynomial because $|f^*|$ may be exponentially large in the number of bits used to encode the capacities. The extra 1 accounts for initialization and the final unsuccessful search. This is the same distinction we encountered for value-indexed knapsack: dependence on a numerical value can be much larger than dependence on its bit length. Unlike the FPTAS preview, here the goal remains an exact optimum.

Choosing shortest augmenting paths by BFS gives the Edmonds-Karp algorithm, whose running time is polynomial in n and m, independent of capacity magnitudes. Here "shortest" means *fewest residual edges*, so ordinary BFS suffices; capacities are not lengths. We will not prove its running-time bound here. With arbitrary real capacities, unrestricted path choices in Ford-Fulkerson need not terminate; the BFS variant avoids this issue.

Theorem (augmenting-path optimality). If the residual graph G_f contains no s-t path, then f is a maximum flow.

To prove this statement and obtain an optimality certificate, we introduce cuts.

Proof plan. An augmenting path certifies that the current flow is improvable. If search fails, we want the vertices it reached to provide the opposite certificate: a cut whose capacity equals the current flow value. First we show that every cut gives an upper bound on every flow.

4 Minimum s-t cut

An s-t cut is a partition $(X, V \setminus X)$ with $s \in X$ and $t \notin X$. Its capacity counts only edges directed out of X:

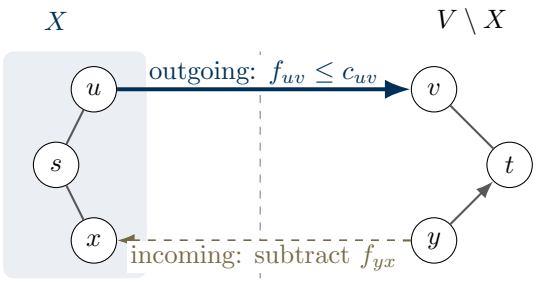
$$c(X, V \setminus X) = \sum_{\substack{(u,v)\in E \\ u\in X,\ v\notin X}} c_{uv}.$$

Observation. For every feasible flow f and every s-t cut,

$$|f| \le c(X, V \setminus X).$$

Thus maximum-flow value is at most minimum-cut capacity.

Reason. Summing flow conservation over vertices of $X \setminus \{s\}$ cancels all flow on edges internal to X. Hence $|f|$ equals flow leaving X minus flow entering X, which is at most the capacity of edges leaving X.



A schematic directed cut. Only outgoing capacities count in its capacity; incoming flow is subtracted when computing net flow across it.

Compare with the MST cut property. For an MST we used the *lightest undirected crossing edge* to justify one greedy choice. For flow we sum *all outgoing capacities* to bound the entire objective. The partition idea is shared, but its role differs.

Theorem (max-flow min-cut). The maximum value of an s-t flow equals the minimum capacity of an s-t cut.

Proof. For the integer capacities assumed here, Ford-Fulkerson terminates by the preceding analysis. Let f be its final flow and let X be the set of vertices reachable from s in G_f. Then $s \in X$ and $t \notin X$.

Consider an original edge (u, v) leaving X. Its forward residual capacity must be zero; otherwise v would also be reachable. Therefore

$$f_{uv} = c_{uv}.$$

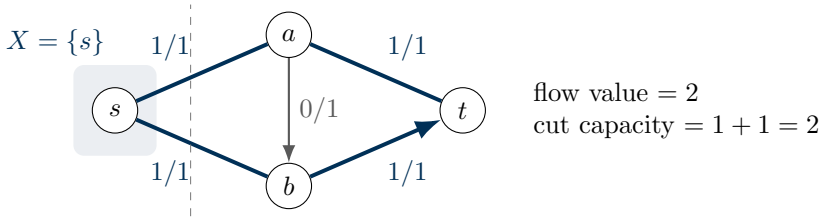
Now consider an original edge (v, u) entering X, where $v \notin X$ and $u \in X$. If $f_{vu} > 0$, its reverse residual edge (u, v) would have positive capacity, again making v reachable. Therefore

$$f_{vu} = 0.$$

The net flow across the cut is consequently

$$|f| = \sum_{\substack{(u,v)\in E \\ u\in X,\ v\notin X}} f_{uv} - \sum_{\substack{(v,u)\in E \\ v\notin X,\ u\in X}} f_{vu} = c(X, V \setminus X).$$

Weak duality gave $|f'| \le c(X, V \setminus X)$ for every feasible flow f'. Our flow reaches this upper bound, so it is maximum and the displayed cut is minimum. □



The final flow in our running example. Both edges leaving s are saturated, so only s is reachable in the residual graph. This cut certifies optimality; the picture shows original edges labeled by flow/capacity.

Returning to bipartite matching. Starting from zero with integer capacities, every augmentation is integral. Our algorithm therefore produces an integral maximum flow. Applied to the unit-capacity matching network, its flow-1 middle edges give a maximum matching, as promised earlier.

Exercise. Explain how the proof gives a short, checkable certificate that a proposed flow is maximum. What data would you hand to someone who did not run the algorithm?

5 Part I synthesis

The first unit has developed three broad algorithmic lenses:

- **divide and conquer:** split, recurse, and combine;
- **dynamic programming:** define overlapping states and reuse their values;
- **graph algorithms:** exploit reachability, cuts, paths, and local exchange structure.

For every algorithm, we paired a construction with a proof invariant and an explicit running-time analysis.

The graph algorithms illustrate two ways to justify progress. Prim makes irrevocable choices using an exchange argument. Matching and flow revise earlier choices along augmenting paths, and a failed search certifies optimality. Finding a solution and explaining why it is optimal are both parts of algorithm design.

Looking ahead to linear programming. The flow constraints are linear equalities and inequalities, and the flow value is a linear objective. The statement that every feasible flow is bounded by every cut is an example of *weak duality*; equality at the optimum previews the duality theory we will develop in Part II.

Exercise. For each of Prim, bipartite matching, and maximum flow, state the maintained invariant, the operation that makes progress, and the argument or certificate used to prove optimality. Which algorithms can undo an earlier choice?